

## Status Player

Chi sta giocando adesso, a cosa, e con quanti punti

### 1. Il problema del punteggio

La richiesta era semplice: *sul pannello devono comparire il nome, il gioco e il punteggio dei giocatori su Batocera*. Due terzi sono facili. Il punteggio no, e vale la pena dire perché prima di dire come.

**Batocera non ha un punteggio.** Ogni gioco tiene il suo dentro la RAM dell'emulatore, a un indirizzo che cambia da titolo a titolo, e non lo scrive da nessuna parte quando si spegne. Chiedere "quanti punti ha fatto" a una macchina che sta facendo girare Super Mario World non è una domanda difficile: è una domanda a cui non esiste una risposta generale. I pochi giochi arcade che salvano gli hiscore su file lo fanno ciascuno a modo suo, e restano su quel disco.

Ce n'è uno solo che vale per tutti i giochi e per tutte le persone: **RetroAchievements**. Batocera lo integra già — *Impostazioni* → *Achievements* — e il suo punteggio ha una proprietà che nessun hiscore ha: è confrontabile. Fra uno che gioca a Sonic e uno che gioca a Metal Slug, i punti dicono qualcosa; i loro punteggi interni non dicono niente.

### 2. Le due sorgenti

|                          | Da dove                                  | Cosa vede                                                | Cosa serve                                                         |
|--------------------------|------------------------------------------|----------------------------------------------------------|--------------------------------------------------------------------|
| <b>RetroAchievements</b> | l'API pubblica, interrogata dal pannello | punti, gioco in corso, cosa sta facendo dentro il gioco  | il nickname dell'amico e una chiave API                            |
| <b>Agente Batocera</b>   | uno script sulla macchina dell'amico     | qualunque gioco, anche senza cheevos, e quando si smette | copiarlo su ogni macchina, e che quella macchina veda il Raspberry |

Si sommano nello stesso registro, e su una persona che ha tutte e due c'è **una riga sola**: il gioco lo dice l'agente, che lo sa per certo, e i punti RetroAchievements, che è l'unico a saperli.

La prima non chiede niente agli amici e funziona anche se sono dall'altra parte d'Italia. La seconda copre i giochi senza cheevos, ma vive in rete locale. Per la maggior parte delle persone la prima basta.

### 3. Sul pannello

Due modi di comparire, perché rispondono a due domande diverse.

**L'avviso** — è *appena successo qualcosa*. Un amico ha avviato un gioco: il pannello lo dice per qualche secondo e poi torna a quello che stava facendo.

|                   |      |        |  |
|-------------------|------|--------|--|
| -----             |      |        |  |
| GILLO ha avviato  |      |        |  |
| Super Mario World | SNES | 12.480 |  |

**Il giro** — *chi c'è in questo momento*. Ogni tanto, una schermata per ciascuno di quelli che stanno giocando.

|                    |        |
|--------------------|--------|
| IN GIOCO           | 1/2    |
| GILLO              | 12.480 |
| World 4-2, 3 lives | +25    |

Il `+25` è quello che rende viva una schermata altrimenti ferma: i punti guadagnati **da quando ha cominciato quella partita**. E la riga in basso, quando c'è, non è il titolo ma la *rich presence* — dove è arrivato, quante vite gli restano — che è la cosa più interessante che l'API sappia dire.

Priorità 62: sopra i satelliti, sotto le notifiche di Home Assistant. Un amico che comincia a giocare è un fatto del momento come il passaggio di un aereo, ma non è un allarme, e soprattutto **non interrompe chi sta giocando al pannello**: quello sta giocando davvero.

I due si spengono separatamente dalla pagina. Chi vuole solo sapere quando qualcuno comincia tiene l'avviso; chi vuole il tabellone tiene il giro.

## 4. RetroAchievements

Serve una **chiave web API**: sta nel tuo profilo su `retroachievements.org`, nella sezione Keys. È personale, quindi si comporta come la password del broker MQTT — **non esce nella configurazione esportata**. Dopo un ripristino si riscrive una volta sola.

Poi l'elenco degli amici: nome da mostrare sul pannello e nickname RetroAchievements. Il nome lo scegli tu, ed è quello che si legge da tre metri; il nickname deve essere esatto.

|                          |                                    |
|--------------------------|------------------------------------|
| Ogni quanto si chiede    | 120 secondi di serie, da 30 a 3600 |
| Quanti amici             | fino a 12                          |
| Dopo quanto uno sparisce | 10 minuti di silenzio              |

I due minuti non sono un numero a caso: una partita dura più di due minuti, quindi non se ne perde nessuna, e sono trenta richieste all'ora per amico invece di centoventi. L'API è gratuita e di qualcun altro; il tetto di dodici amici serve a che un giro non duri più dell'intervallo, perché a quel punto il pannello racconterebbe cose vecchie credendole fresche.

**Su quando una persona "sta giocando"**. L'ora la dice l'API, non il pannello: se l'ultima presenza di qualcuno risale a due ore fa, quella persona non sta giocando, per quanto spesso glielo si chieda. E quando l'API non la dice, una risposta identica ripetuta ogni due minuti non è la prova che qualcuno stia giocando — è la prova che il profilo è fermo.

Il pulsante **Prova adesso** fa subito un giro e scrive com'è andata: è il modo di scoprire che la chiave è sbagliata *adesso*, invece che fra due minuti e in silenzio.

## 5. L'agente su Batocera

EmulationStation esegue gli script che trova in

`/userdata/system/configs/emulationstation/scripts/<evento>/` e passa tre argomenti: il sistema, il file della rom, il nome del gioco. Lo stesso file sta in `game-start` e in `game-end`, e capisce da solo quale dei due è guardando la cartella da cui è stato eseguito: così non ci sono due script da tenere allineati.

La pagina Status Player detta le righe esatte da incollare, **con dentro il tuo indirizzo e il tuo segreto**, perché un'istruzione che dice `<indirizzo del pannello>` è un'istruzione che qualcuno copierà così com'è. In sostanza:

```
curl -o /userdata/system/statusplayer.sh http://PANNELLO:8080/static/statusplayer.sh
chmod +x /userdata/system/statusplayer.sh
ln -sf /userdata/system/statusplayer.sh \
    /userdata/system/configs/emulationstation/scripts/game-start/statusplayer.sh
ln -sf /userdata/system/statusplayer.sh \
    /userdata/system/configs/emulationstation/scripts/game-end/statusplayer.sh
batocera-save-overlay
```

e un file `/userdata/system/statusplayer.conf` con tre righe: `PANNELLO`, `TOKEN`, `NOME`.

**Il segreto non è formale.** Senza, l'indirizzo che riceve gli eventi accetterebbe

qualunque messaggio da chiunque sia sulla rete di casa: un cartello «scrivi qui». Si genera dalla pagina, è lo stesso per tutti gli amici, e viaggia in chiaro come tutto il resto della web UI — che è il motivo per cui questo servizio è pensato per la rete di casa e non per internet.

L'agente manda tre cose: il nome che hai scelto, il sistema e il titolo del gioco. Niente percorsi di file, niente indirizzi, niente altro. E ha due secondi di pazienza: se il pannello è spento, il gioco parte lo stesso.

Il segreto, al contrario della chiave API, **resta** nella configurazione esportata: è un segreto di questo pannello, e se sparisse in un ripristino tutti gli agenti già installati smetterebbero di parlare senza che nessuno capisca perché.

## 6. Come è stato provato

---

`test_statusplayer.py`, 76 controlli. Quelli che contano davvero:

- **un titolo si chiede una volta sola**: il numero del gioco diventa un titolo, e il titolo resta su disco. Un titolo non cambia mai, e chiederlo ogni due minuti per sempre a un servizio gratuito sarebbe maleducazione;
- **la chiave API non torna mai in una pagina HTML** e non esce in una configurazione esportata, e salvare la pagina con la casella vuota non la cancella;
- **senza segreto non si accetta niente**, con il segreto sbagliato nemmeno;
- **un nickname impossibile non si salva**: finirebbe in una query string, e il giro fallirebbe ogni due minuti per sempre;
- **un profilo fermo da due ore non compare come in gioco**, che è il difetto in cui cade chi mette l'ora di adesso su un dato che l'ora ce l'ha già.